

Homework 9 Solutions

April 14, 2026

1. Problem 1

- (a) Let $s[i, j)$ be the substring of s from index i to index $j - 1$ (ie $s[i]s[i + 1] \cdots s[j - 1]$ and is empty when $i = j$). For $0 \leq i \leq j \leq n$, consider the subproblem of determining whether the substring $s[i, j)$ is a palindrome. If $j - i \geq 2$, then $s[i, j)$ is a palindrome if and only if the first and last characters match and the interior is a palindrome; i.e., $s[i] = s[j - 1]$ and $s[i + 1, j - 1)$ is a palindrome. The base cases are that the empty substring $s[i, i)$ is a palindrome for all i , and every length-1 substring $s[i, i + 1)$ is also a palindrome. Once we know this for all subintervals, the length of the longest palindromic substring is the maximum length $j - i$ over all pairs (i, j) such that $s[i, j)$ is a palindrome.

NB: You could also define $s[i, j]$ to be inclusive then the base case is any length 1 string $s[i, i]$ is a palindrome and any length 2 string $s[i, i + 1]$ is a palindrome iff $s[i] = s[i + 1]$. I just like making the empty set a base case in general.

- (b) Let $P[i][j] \in \{\text{True}, \text{False}\}$ indicate whether $s[i, j)$ is a palindrome (so $0 \leq i \leq j \leq n$).

$$P[i][j] = \begin{cases} \text{True}, & j = i, \\ \text{True}, & j = i + 1, \\ (s[i] = s[j - 1]) \wedge P[i + 1][j - 1], & j \geq i + 2. \end{cases}$$

The answer is

$$\max\{j - i : 0 \leq i \leq j \leq n \text{ and } P[i][j] = \text{True}\}.$$

NB: that you could also define $P[i, \ell]$ to be if the substring of length ℓ starting at i is a palindrome, and the recurrence would be similar.

(c) 1: **function** LONGESTPALSUBSTRINGLENGTH(s)
2: $n \leftarrow |s|$
3: initialize $P[0..n][0..n] \leftarrow \text{False}$
4: **for** $i \leftarrow 0$ **to** n **do**
5: $P[i][i] \leftarrow \text{True}$
6: **end for**
7: **for** $L \leftarrow 1$ **to** n **do** $\triangleright$ length $L = j - i$
8: **for** $i \leftarrow 0$ **to** $n - L$ **do**
9: $j \leftarrow i + L$
10: **if** $L \leq 1$ **then**
11: $P[i][j] \leftarrow \text{True}$
12: **else if** $s[i] = s[j - 1]$ **then**
13: $P[i][j] \leftarrow P[i + 1][j - 1]$
14: **else**
15: $P[i][j] \leftarrow \text{False}$
16: **end if**
17: **end for**
18: **end for**
19: **return** $\max\{j - i : 0 \leq i \leq j \leq n \text{ and } P[i][j] = \text{True}\}$
20: **end function**

(d) We filling by diagonals of increasing length $L = j - i$ (so from the bottom-left to the top-right).

$P[i][j]$	$j = 0$	1	2	3	4	5	6	7	8
$i = 0 (m)$	True	True	False	False	False	False	False	False	False
1 (a)		True	True	False	False	False	False	False	False
2 (d)			True	True	True	False	False	False	False
3 (d)				True	True	False	False	False	True
4 (e)					True	True	False	True	False
5 (n)						True	True	False	False
6 (e)							True	True	False
7 (d)								True	True
8 (end)									True

From the table, the longest palindromic substring has length 5, which corresponds to “dened”

2. Problem 2

- (a) Define a subproblem $F(i, j)$ to be the side length of the largest all-zero square submatrix whose bottom-right corner is at (i, j) . If the entry at (i, j) is True, then no all-zero square can end there, so $F(i, j) = 0$. If the entry at (i, j) is False, then the largest square ending at (i, j) can be formed by extending a smaller square by one row and one column; the

extension is limited by the largest squares ending immediately above with bottom left corner at $(i-1, j)$, the left square $(i, j-1)$, and the top-left $(i-1, j-1)$.

The overall answer is $\max_{i,j} F(i, j)$.

(b)

$$F(i, j) = \begin{cases} 0, & A[i][j] = 1, \\ 1, & A[i][j] = 0 \text{ and } (i = 0 \text{ or } j = 0), \\ 1 + \min\{F(i-1, j), F(i, j-1), F(i-1, j-1)\}, & A[i][j] = 0 \text{ and } i, j > 0. \end{cases}$$

(c) 1: **function** LARGESTZEROSQUARE(A)
2: $(n, m) \leftarrow$ dimensions of A
3: initialize $\text{memo}[0..n-1][0..m-1] \leftarrow -1$
4: **function** $F(i, j)$
5: **if** $i < 0$ **or** $j < 0$ **then**
6: **return** 0
7: **end if**
8: **if** $\text{memo}[i][j] \neq -1$ **then**
9: **return** $\text{memo}[i][j]$
10: **end if**
11: **if** $A[i][j] = 1$ **then**
12: $\text{memo}[i][j] \leftarrow 0$
13: **else**
14: $\text{memo}[i][j] \leftarrow 1 + \min(F(i-1, j), F(i, j-1), F(i-1, j-1))$
15: **end if**
16: **return** $\text{memo}[i][j]$
17: **end function**
18: **return** $\max\{F(i, j) : 0 \leq i < n, 0 \leq j < m\}$
19: **end function**

3. Problem 3

- (a) Let the input be symbols $x_1, \dots, x_n \in \{a, b, c\}$. For any interval $[i..j]$, whether the expression $x_i \times x_{i+1} \times \dots \times x_j$ can evaluate to a particular result in $\{a, b, c\}$ depends on where we place the final split. If the final operation combines a left hand expression on $[i..k]$ and a right hand subexpression on $[k+1..j]$, then the result is determined by applying $\times$ to a possible left value and a possible right value. Thus the subproblem on $[i..j]$ reduces to smaller subproblems on $[i..k]$ and $[k+1..j]$. Concretely, we need to determine if there are some possible ways to evaluate the left and right subexpressions to values p and q such that $p \times q$ is the desired result.

NB: Notice that $a \times b \times c = (a \times b) \times (c)$.

- (b) Let $DP[i][j][r]$ be a boolean that is true iff the subexpression from symbol i to j can evaluate to result $r \in \{a, b, c\}$.

Base case:

$$DP[i][i][r] = \begin{cases} \text{True}, & r = x_i, \\ \text{False}, & r \neq x_i. \end{cases}$$

For $i < j$:

$$DP[i][j][r] = \bigvee_{k=i}^{j-1} \bigvee_{p \in \{a,b,c\}} \bigvee_{q \in \{a,b,c\}} \left(\underbrace{DP[i][k][p]}_{\text{left subexpression can be } p} \wedge \underbrace{DP[k+1][j][q]}_{\text{right subexpression can be } q} \wedge \underbrace{(p \times q = r)}_{\text{combining yields } r} \right).$$

(c) 1: **function** CANEVALUATEToA($x[1..n]$)
2: initialize $DP[1..n][1..n][a, b, c] \leftarrow \text{False}$
3: **for** $i \leftarrow 1$ **to** n **do**
4: $DP[i][i][x_i] \leftarrow \text{True}$
5: **end for**
6: **for** $d \leftarrow 1$ **to** $n - 1$ **do**
7: **for** $i \leftarrow 1$ **to** $n - d$ **do**
8: $j \leftarrow i + d$
9: **for** $k \leftarrow i$ **to** $j - 1$ **do**
10: **for all** $p \in \{a, b, c\}$ **do**
11: **for all** $q \in \{a, b, c\}$ **do**
12: $DP[i][j][p \times q] \leftarrow DP[i][j][p \times q] \vee (DP[i][k][p] \wedge DP[k+1][j][q])$
13: **end for**
14: **end for**
15: **end for**
16: **end for**
17: **end for**
18: **return** $DP[1][n][a]$
19: **end function**

- (d) In the table I am recording the set of everything reachable from the interval, but you could also just record a boolean for each of a, b, c and the logic would be the same. Fill in in increasing length

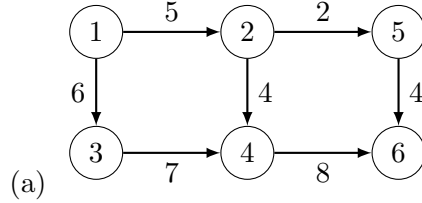
$S[i, j]$	$j = 1$	2	3	4	5	6	7
$i = 1 (a)$	$\{a\}$	$\{b\}$	$\{a, c\}$	$\{a, c\}$	$\{a, c\}$	$\{a, b\}$	$\{a, b, c\}$
2 (b)		$\{b\}$	$\{c\}$	$\{c\}$	$\{c\}$	$\{a, b\}$	$\{b, c\}$
3 (a)			$\{a\}$	$\{a\}$	$\{a\}$	$\{b\}$	$\{a, b\}$
4 (c)				$\{c\}$	$\{c\}$	$\{a\}$	$\{b, c\}$
5 (c)					$\{c\}$	$\{a\}$	$\{b, c\}$
6 (a)						$\{a\}$	$\{b\}$
7 (b)							$\{b\}$

- (e) Backtrack directly from the filled table. Start from the state $(i, j, r) = (1, n, a)$.

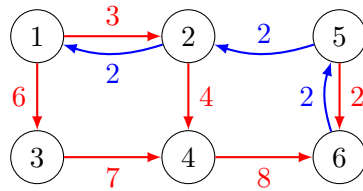
If $i = j$, the expression is just the single symbol x_i and we stop. Otherwise (so $i < j$), since $DP[i][j][r] = \text{True}$ the recurrence says that there exist some split point $k \in \{i, \dots, j-1\}$ and values $p, q \in \{a, b, c\}$ such that $DP[i][k][p] = \text{True}$, $DP[k+1][j][q] = \text{True}$, and $p \times q = r$. Find any such triple (k, p, q) by scanning over k , p , and q . Then recursively recover a the parenthesization of $x_i \times \dots \times x_k$ that evaluates to p and one of $x_{k+1} \times \dots \times x_j$ that evaluates to q , and combine them.

4. Problem 4

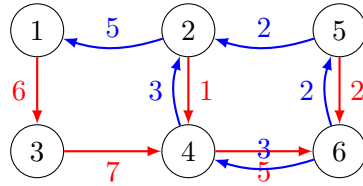
I color coded the arcs in the original flow graph red and the backwards arcs in the residual graph blue. This is not needed but it made it easier for me to keep track of the forward and backward edges and to read.



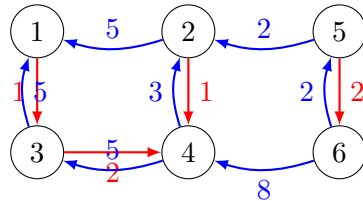
send 2 along $1 \rightarrow 2 \rightarrow 5 \rightarrow 6$:



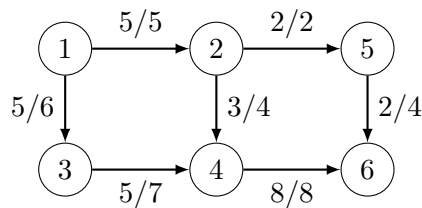
send 3 along $1 \rightarrow 2 \rightarrow 4 \rightarrow 6$



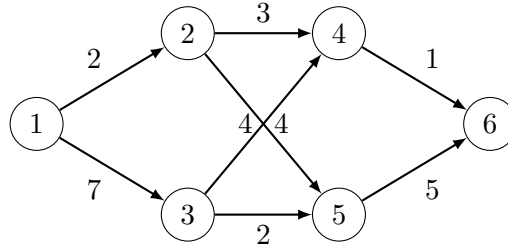
send $\Delta = 5$ along $1 \rightarrow 3 \rightarrow 4 \rightarrow 6$



Max flow of 10.

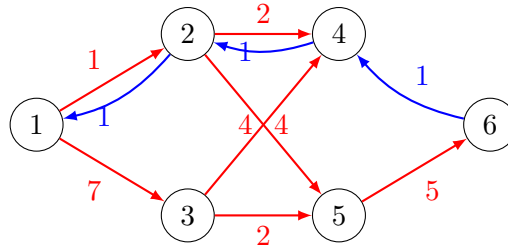


The vertices reachable from 1 are $S = \{1, 2, 3, 4\}$, so we can take $T = \{5, 6\}$, so this is the max cut. The cut capacity is $c(S, T) = c(2 \rightarrow 5) + c(4 \rightarrow 6) = 2 + 8 = 10$.

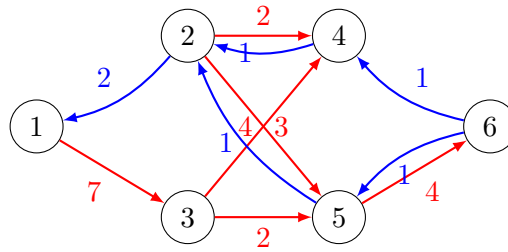


(b)

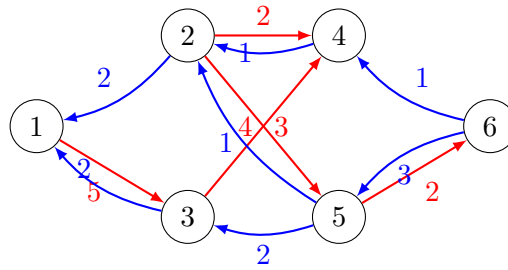
send 1 along $1 \rightarrow 2 \rightarrow 4 \rightarrow 6$:



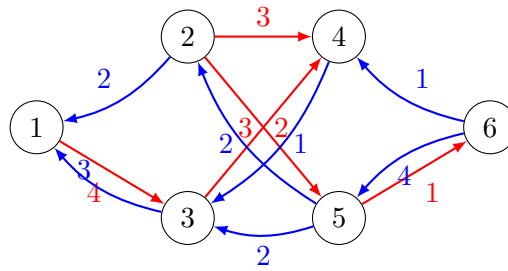
send 1 along $1 \rightarrow 2 \rightarrow 5 \rightarrow 6$:



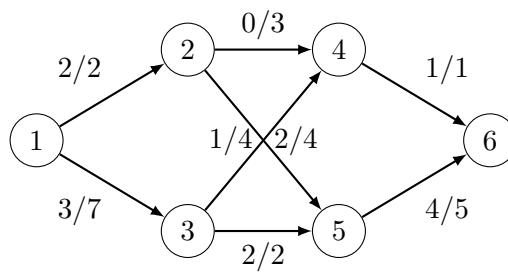
send 2 along $1 \rightarrow 3 \rightarrow 5 \rightarrow 6$:



send 1 along $1 \rightarrow 3 \rightarrow 4 \rightarrow 2 \rightarrow 5 \rightarrow 6$:



Max flow of 5.



The vertices reachable from 1 are $S = \{1, 3, 4\}$, so we can take $T = \{2, 5, 6\}$ for our cut. The cut capacity is $c(S, T) = c(1 \rightarrow 2) + c(3 \rightarrow 5) + c(4 \rightarrow 6) = 2 + 2 + 1 = 5$.